

DSC550Week9.3 - Milestone3

August 2, 2025

1 Content

1. Introduction
 - What is Customer Churn?
 - How can customer churn be reduced?
 - Objectives
2. Loading libraries and data
3. Understanding the data
4. Visualize missing values
5. Data Manipulation
6. Data Visualization
7. Data Preprocessing
 - Standardizing numeric attributes
8. Machine Learning Model Evaluations and Predictions
 - KNN
 - SVC
 - Logistic Regression
 - Decision Tree Classifier
 - Gradient Boosting Classifier

1. Introduction

What is Customer Churn? Customer churn is defined as when customers or subscribers discontinue doing business with a firm or service.

Customers in the telecom industry can choose from a variety of service providers and actively switch from one to the next. The telecommunications business has an annual churn rate of 15-25 percent in this highly competitive market.

Individualized customer retention is tough because most firms have a large number of customers and can't afford to devote much time to each of them. The costs would be too great, outweighing the additional revenue. However, if a corporation could forecast which customers are likely to leave ahead of time, it could focus customer retention efforts only on these "high risk" clients. The ultimate goal is to expand its coverage area and retrieve more customers loyalty. The core to succeed in this market lies in the customer itself.

Customer churn is a critical metric because it is much less expensive to retain existing customers than it is to acquire new customers.

As a result, by addressing churn, these businesses may not only preserve their market position, but also grow and thrive. More customers they have in their network, the lower the cost of initiation

and the larger the profit. As a result, the company's key focus for success is reducing client attrition and implementing effective retention strategy.

```
[112]: pip install missingno
```

```
Requirement already satisfied: missingno in /opt/anaconda3/lib/python3.12/site-
packages (0.5.2)
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.12/site-
packages (from missingno) (1.26.4)
Requirement already satisfied: matplotlib in /opt/anaconda3/lib/python3.12/site-
packages (from missingno) (3.8.4)
Requirement already satisfied: scipy in /opt/anaconda3/lib/python3.12/site-
packages (from missingno) (1.13.1)
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python3.12/site-
packages (from missingno) (0.13.2)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno) (1.2.0)
Requirement already satisfied: cyclor>=0.10 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno)
(0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno)
(4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno) (1.4.4)
Requirement already satisfied: packaging>=20.0 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno) (23.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.12/site-
packages (from matplotlib->missingno) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/anaconda3/lib/python3.12/site-packages (from matplotlib->missingno)
(2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in
/opt/anaconda3/lib/python3.12/site-packages (from seaborn->missingno) (2.2.2)
Requirement already satisfied: pytz>=2020.1 in
/opt/anaconda3/lib/python3.12/site-packages (from
pandas>=1.2->seaborn->missingno) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in
/opt/anaconda3/lib/python3.12/site-packages (from
pandas>=1.2->seaborn->missingno) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/site-
packages (from python-dateutil>=2.7->matplotlib->missingno) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[113]: import pandas as pd
import numpy as np
```

```

import missingno as msno
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

```

2. Loading libraries and data

```

[115]: from sklearn.preprocessing import StandardScaler
       from sklearn.preprocessing import LabelEncoder

       from sklearn.tree import DecisionTreeClassifier
       from sklearn.ensemble import RandomForestClassifier
       from sklearn.naive_bayes import GaussianNB
       from sklearn.neighbors import KNeighborsClassifier
       from sklearn.svm import SVC
       from sklearn.neural_network import MLPClassifier
       from sklearn.ensemble import AdaBoostClassifier
       from sklearn.ensemble import GradientBoostingClassifier
       from sklearn.ensemble import ExtraTreesClassifier
       from sklearn.linear_model import LogisticRegression
       from sklearn.model_selection import train_test_split
       from sklearn.metrics import accuracy_score
       from xgboost import XGBClassifier
       from sklearn import metrics
       from sklearn.metrics import roc_curve
       from sklearn.metrics import recall_score, confusion_matrix, precision_score, \
         f1_score, accuracy_score, classification_report

```

```

[116]: #loading data
       df = pd.read_csv('./WA_Fn-UseC_-Telco-Customer-Churn.csv')

```

3. Understanding the data

Each row represents a customer, each column contains customer's attributes described on the column Metadata.

```

[119]: df.head()

```

```

[119]:  customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  7590-VHVEG  Female                0      Yes           No         1           No
1  5575-GNVDE   Male                0      No            No        34           Yes
2  3668-QPYBK   Male                0      No            No         2           Yes
3  7795-CFOCW   Male                0      No            No        45           No
4  9237-HQITU   Female              0      No            No         2           Yes

```

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No phone service		DSL	No	...	No
1	No		DSL	Yes	...	Yes
2	No		DSL	Yes	...	No
3	No phone service		DSL	Yes	...	Yes
4	No	Fiber optic		No	...	No

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month		Yes
1	No	No	No	One year		No
2	No	No	No	Month-to-month		Yes
3	Yes	No	No	One year		No
4	No	No	No	Month-to-month		Yes

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]

The data set includes information about: * Customers who left within the last month – the column is called Churn

- **Services that each customer has signed up for** – phone, multiple lines, internet, online security, online backup, device protection, tech support, and streaming TV and movies
- **Customer account information** - how long they've been a customer, contract, payment method, paperless billing, monthly charges, and total charges
- **Demographic info about customers** – gender, age range, and if they have partners and dependents

```
[121]: df.shape
```

```
[121]: (7043, 21)
```

```
[122]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null   object
1   gender                7043 non-null   object
2   SeniorCitizen         7043 non-null   int64
3   Partner              7043 non-null   object
```

```

4   Dependents      7043 non-null  object
5   tenure          7043 non-null  int64
6   PhoneService    7043 non-null  object
7   MultipleLines   7043 non-null  object
8   InternetService 7043 non-null  object
9   OnlineSecurity  7043 non-null  object
10  OnlineBackup     7043 non-null  object
11  DeviceProtection 7043 non-null  object
12  TechSupport      7043 non-null  object
13  StreamingTV      7043 non-null  object
14  StreamingMovies  7043 non-null  object
15  Contract         7043 non-null  object
16  PaperlessBilling 7043 non-null  object
17  PaymentMethod    7043 non-null  object
18  MonthlyCharges   7043 non-null  float64
19  TotalCharges     7043 non-null  object
20  Churn            7043 non-null  object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

```

```
[127]: df.columns.values
```

```

[127]: array(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
            'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
            'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
            'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
            'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
            'TotalCharges', 'Churn'], dtype=object)

```

```
[132]: df.dtypes
```

```

[132]: customerID      object
gender                object
SeniorCitizen         int64
Partner               object
Dependents            object
tenure                int64
PhoneService          object
MultipleLines         object
InternetService       object
OnlineSecurity        object
OnlineBackup          object
DeviceProtection      object
TechSupport           object
StreamingTV           object
StreamingMovies       object
Contract              object
PaperlessBilling      object

```

```

PaymentMethod      object
MonthlyCharges     float64
TotalCharges       object
Churn              object
dtype: object

```

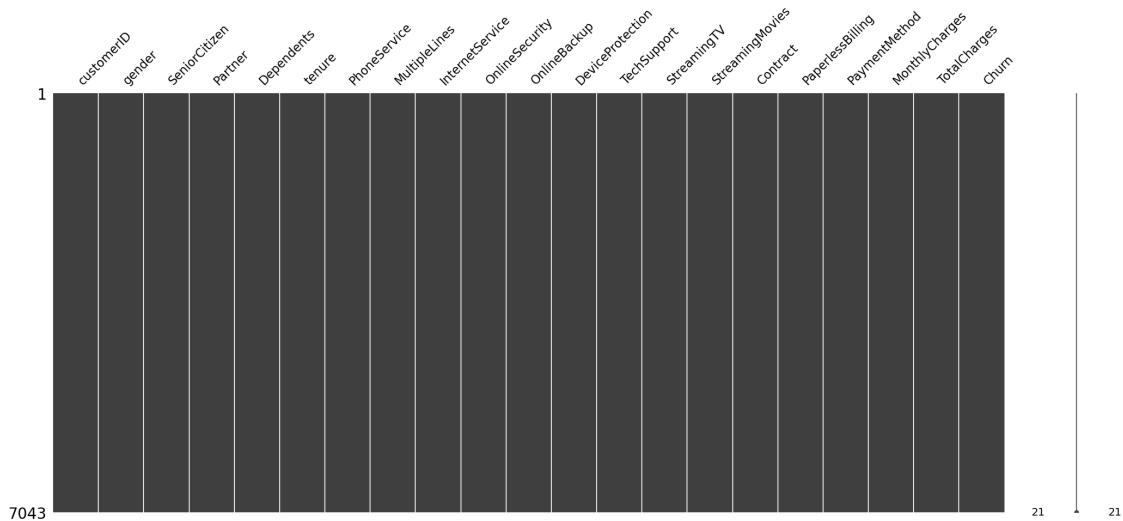
- The target the we will use to guide the exploration is **Churn**

4. Visualize missing values

```

[136]: # Visualize missing values as a matrix
msno.matrix(df);

```



Using this matrix we can very quickly find the pattern of missingness in the dataset. *
 From the above visualisation we can observe that it has no peculiar pattern that stands out. In fact there is no missing data.

5. Data Manipulation

```

[140]: df = df.drop(['customerID'], axis = 1)
df.head()

```

```

[140]:   gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  Female                0     Yes         No         1           No
1   Male                0     No          No        34           Yes
2   Male                0     No          No         2           Yes
3   Male                0     No          No        45           No
4  Female                0     No          No         2           Yes

      MultipleLines  InternetService  OnlineSecurity  OnlineBackup  \
0  No phone service              DSL                No           Yes

```

1	No	DSL	Yes	No
2	No	DSL	Yes	Yes
3	No phone service	DSL	Yes	No
4	No	Fiber optic	No	No

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	No	No	No	No	Month-to-month	
1	Yes	No	No	No	One year	
2	No	No	No	No	Month-to-month	
3	Yes	Yes	No	No	One year	
4	No	No	No	No	Month-to-month	

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	\
0	Yes	Electronic check	29.85	29.85	
1	No	Mailed check	56.95	1889.5	
2	Yes	Mailed check	53.85	108.15	
3	No	Bank transfer (automatic)	42.30	1840.75	
4	Yes	Electronic check	70.70	151.65	

Churn	
0	No
1	No
2	Yes
3	No
4	Yes

- On deep analysis, we can find some indirect missingness in our data (which can be in form of blankspaces). Let's see that!

```
[143]: df['TotalCharges'] = pd.to_numeric(df.TotalCharges, errors='coerce')
df.isnull().sum()
```

```
[143]: gender          0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
```

```

PaymentMethod      0
MonthlyCharges     0
TotalCharges       11
Churn              0
dtype: int64

```

- Here we see that the TotalCharges has 11 missing values. Let's check this data.

```
[146]: df[np.isnan(df['TotalCharges'])]
```

```
[146]:
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
488	Female	0	Yes	Yes	0	No	
753	Male	0	No	Yes	0	Yes	
936	Female	0	Yes	Yes	0	Yes	
1082	Male	0	Yes	Yes	0	Yes	
1340	Female	0	Yes	Yes	0	No	
3331	Male	0	Yes	Yes	0	Yes	
3826	Male	0	Yes	Yes	0	Yes	
4380	Female	0	Yes	Yes	0	Yes	
5218	Male	0	Yes	Yes	0	Yes	
6670	Female	0	Yes	Yes	0	Yes	
6754	Male	0	No	Yes	0	Yes	

	MultipleLines	InternetService	OnlineSecurity	\
488	No phone service	DSL	Yes	
753	No	No	No internet service	
936	No	DSL	Yes	
1082	Yes	No	No internet service	
1340	No phone service	DSL	Yes	
3331	No	No	No internet service	
3826	Yes	No	No internet service	
4380	No	No	No internet service	
5218	No	No	No internet service	
6670	Yes	DSL	No	
6754	Yes	DSL	Yes	

	OnlineBackup	DeviceProtection	TechSupport	\
488	No	Yes	Yes	
753	No internet service	No internet service	No internet service	
936	Yes	Yes	No	
1082	No internet service	No internet service	No internet service	
1340	Yes	Yes	Yes	
3331	No internet service	No internet service	No internet service	
3826	No internet service	No internet service	No internet service	
4380	No internet service	No internet service	No internet service	
5218	No internet service	No internet service	No internet service	
6670	Yes	Yes	Yes	
6754	Yes	No	Yes	

	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
488	Yes	No	Two year	Yes	
753	No internet service	No internet service	Two year	No	
936	Yes	Yes	Two year	No	
1082	No internet service	No internet service	Two year	No	
1340	Yes	No	Two year	No	
3331	No internet service	No internet service	Two year	No	
3826	No internet service	No internet service	Two year	No	
4380	No internet service	No internet service	Two year	No	
5218	No internet service	No internet service	One year	Yes	
6670	Yes	No	Two year	No	
6754	No	No	Two year	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
488	Bank transfer (automatic)	52.55	NaN	No
753	Mailed check	20.25	NaN	No
936	Mailed check	80.85	NaN	No
1082	Mailed check	25.75	NaN	No
1340	Credit card (automatic)	56.05	NaN	No
3331	Mailed check	19.85	NaN	No
3826	Mailed check	25.35	NaN	No
4380	Mailed check	20.00	NaN	No
5218	Mailed check	19.70	NaN	No
6670	Mailed check	73.35	NaN	No
6754	Bank transfer (automatic)	61.90	NaN	No

- It can also be noted that the Tenure column is 0 for these entries even though the Monthly-Charges column is not empty.

Let's see if there are any other 0 values in the tenure column.

```
[149]: df[df['tenure'] == 0].index
```

```
[149]: Index([488, 753, 936, 1082, 1340, 3331, 3826, 4380, 5218, 6670, 6754],
dtype='int64')
```

- There are no additional missing values in the Tenure column.

Let's delete the rows with missing values in Tenure columns since there are only 11 rows and deleting them will not affect the data.

```
[152]: df.drop(labels=df[df['tenure'] == 0].index, axis=0, inplace=True)
df[df['tenure'] == 0].index
```

```
[152]: Index([], dtype='int64')
```

To solve the problem of missing values in TotalCharges column, I decided to fill it with the mean of TotalCharges values.

```
[155]: df.fillna(df["TotalCharges"].mean())
```

```
[155]:
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	Female	0	Yes	No	1	No	
1	Male	0	No	No	34	Yes	
2	Male	0	No	No	2	Yes	
3	Male	0	No	No	45	No	
4	Female	0	No	No	2	Yes	
...	...	...	...	...	...	...	
7038	Male	0	Yes	Yes	24	Yes	
7039	Female	0	Yes	Yes	72	Yes	
7040	Female	0	Yes	Yes	11	No	
7041	Male	1	Yes	No	4	Yes	
7042	Male	0	No	No	66	Yes	

	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	\
0	No phone service	DSL	No	Yes	
1	No	DSL	Yes	No	
2	No	DSL	Yes	Yes	
3	No phone service	DSL	Yes	No	
4	No	Fiber optic	No	No	
...	...	...	...	...	
7038	Yes	DSL	Yes	No	
7039	Yes	Fiber optic	No	Yes	
7040	No phone service	DSL	Yes	No	
7041	Yes	Fiber optic	No	No	
7042	No	Fiber optic	Yes	No	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	No	No	No	No	Month-to-month	
1	Yes	No	No	No	One year	
2	No	No	No	No	Month-to-month	
3	Yes	Yes	No	No	One year	
4	No	No	No	No	Month-to-month	
...	...	...	...	...	...	
7038	Yes	Yes	Yes	Yes	One year	
7039	Yes	No	Yes	Yes	One year	
7040	No	No	No	No	Month-to-month	
7041	No	No	No	No	Month-to-month	
7042	Yes	Yes	Yes	Yes	Two year	

	PaperlessBilling	PaymentMethod	MonthlyCharges	\
0	Yes	Electronic check	29.85	
1	No	Mailed check	56.95	
2	Yes	Mailed check	53.85	
3	No	Bank transfer (automatic)	42.30	
4	Yes	Electronic check	70.70	

...	...	...	...
7038	Yes	Mailed check	84.80
7039	Yes	Credit card (automatic)	103.20
7040	Yes	Electronic check	29.60
7041	Yes	Mailed check	74.40
7042	Yes	Bank transfer (automatic)	105.65

	TotalCharges	Churn
0	29.85	No
1	1889.50	No
2	108.15	Yes
3	1840.75	No
4	151.65	Yes
...	...	...
7038	1990.50	No
7039	7362.90	No
7040	346.45	No
7041	306.60	Yes
7042	6844.50	No

[7032 rows x 20 columns]

```
[157]: df.isnull().sum()
```

```
[157]: gender                0
       SeniorCitizen        0
       Partner              0
       Dependents           0
       tenure               0
       PhoneService         0
       MultipleLines        0
       InternetService      0
       OnlineSecurity       0
       OnlineBackup         0
       DeviceProtection     0
       TechSupport          0
       StreamingTV          0
       StreamingMovies      0
       Contract             0
       PaperlessBilling     0
       PaymentMethod        0
       MonthlyCharges       0
       TotalCharges         0
       Churn                0
       dtype: int64
```

```
[159]: df["SeniorCitizen"] = df["SeniorCitizen"].map({0: "No", 1: "Yes"})
df.head()
```

```
[159]:
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	Female	No	Yes	No	1	No	
1	Male	No	No	No	34	Yes	
2	Male	No	No	No	2	Yes	
3	Male	No	No	No	45	No	
4	Female	No	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	\
0	No phone service	DSL	No	Yes	
1	No	DSL	Yes	No	
2	No	DSL	Yes	Yes	
3	No phone service	DSL	Yes	No	
4	No	Fiber optic	No	No	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	No	No	No	No	Month-to-month	
1	Yes	No	No	No	One year	
2	No	No	No	No	Month-to-month	
3	Yes	Yes	No	No	One year	
4	No	No	No	No	Month-to-month	

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	\
0	Yes	Electronic check	29.85	29.85	
1	No	Mailed check	56.95	1889.50	
2	Yes	Mailed check	53.85	108.15	
3	No	Bank transfer (automatic)	42.30	1840.75	
4	Yes	Electronic check	70.70	151.65	

	Churn
0	No
1	No
2	Yes
3	No
4	Yes

```
[161]: df["InternetService"].describe(include=['object', 'bool'])
```

```
[161]: count          7032
unique           3
top      Fiber optic
freq           3096
Name: InternetService, dtype: object
```

```
[163]: numerical_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
df[numerical_cols].describe()
```

```
[163]:
```

	tenure	MonthlyCharges	TotalCharges
count	7032.000000	7032.000000	7032.000000
mean	32.421786	64.798208	2283.300441
std	24.545260	30.085974	2266.771362
min	1.000000	18.250000	18.800000
25%	9.000000	35.587500	401.450000
50%	29.000000	70.350000	1397.475000
75%	55.000000	89.862500	3794.737500
max	72.000000	118.750000	8684.800000

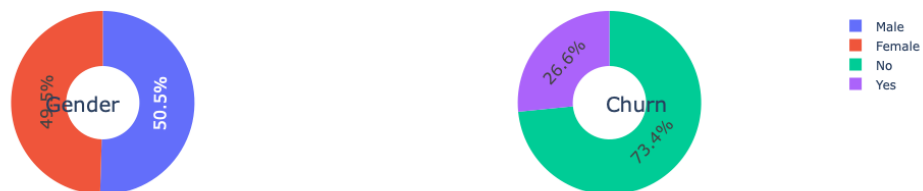
6. Data Visualization

```
[166]: g_labels = ['Male', 'Female']
c_labels = ['No', 'Yes']
# Create subplots: use 'domain' type for Pie subplot
fig = make_subplots(rows=1, cols=2, specs=[[{'type': 'domain'}], {'type':
    ↳ 'domain'}]])
fig.add_trace(go.Pie(labels=g_labels, values=df['gender'].value_counts(),
    ↳ name="Gender"), 1, 1)
fig.add_trace(go.Pie(labels=c_labels, values=df['Churn'].value_counts(),
    ↳ name="Churn"), 1, 2)

# Use `hole` to create a donut-like pie chart
fig.update_traces(hole=.4, hoverinfo="label+percent+name", textfont_size=16)

fig.update_layout(
    title_text="Gender and Churn Distributions",
    # Add annotations in the center of the donut pies.
    annotations=[dict(text='Gender', x=0.16, y=0.5, font_size=20,
    ↳ showarrow=False),
                  dict(text='Churn', x=0.84, y=0.5, font_size=20,
    ↳ showarrow=False)])
fig.show()
```

Gender and Churn Distributions



- 26.6 % of customers switched to another firm.
- Customers are 49.5 % female and 50.5 % male.

```
[169]: df["Churn"][df["Churn"]=="No"].groupby(by=df["gender"]).count()
```

```
[169]: gender
Female    2544
Male      2619
Name: Churn, dtype: int64
```

```
[171]: df["Churn"][df["Churn"]=="Yes"].groupby(by=df["gender"]).count()
```

```
[171]: gender
Female     939
Male       930
Name: Churn, dtype: int64
```

```
[173]: plt.figure(figsize=(6, 6))
labels = ["Churn: Yes", "Churn: No"]
values = [1869, 5163]
labels_gender = ["F", "M", "F", "M"]
sizes_gender = [939, 930, 2544, 2619]
colors = ['#ff6666', '#66b3ff']
colors_gender = ['#c2c2f0', '#ffb3e6', '#c2c2f0', '#ffb3e6']
explode = (0.3, 0.3)
explode_gender = (0.1, 0.1, 0.1, 0.1)
textprops = {"fontsize": 15}
#Plot
plt.pie(values, labels=labels, autopct='%1.1f%%', pctdistance=1.08,
        ↪labeldistance=0.8, colors=colors, startangle=90, frame=True,
        ↪explode=explode, radius=10, textprops=textprops, counterclock=True, )
plt.pie(sizes_gender, labels=labels_gender, colors=colors_gender, startangle=90,
        ↪explode=explode_gender, radius=7, textprops=textprops, counterclock=True, )
#Draw circle
centre_circle = plt.Circle((0,0),5,color='black', fc='white',linewidth=0)
fig = plt.gcf()
fig.gca().add_artist(centre_circle)

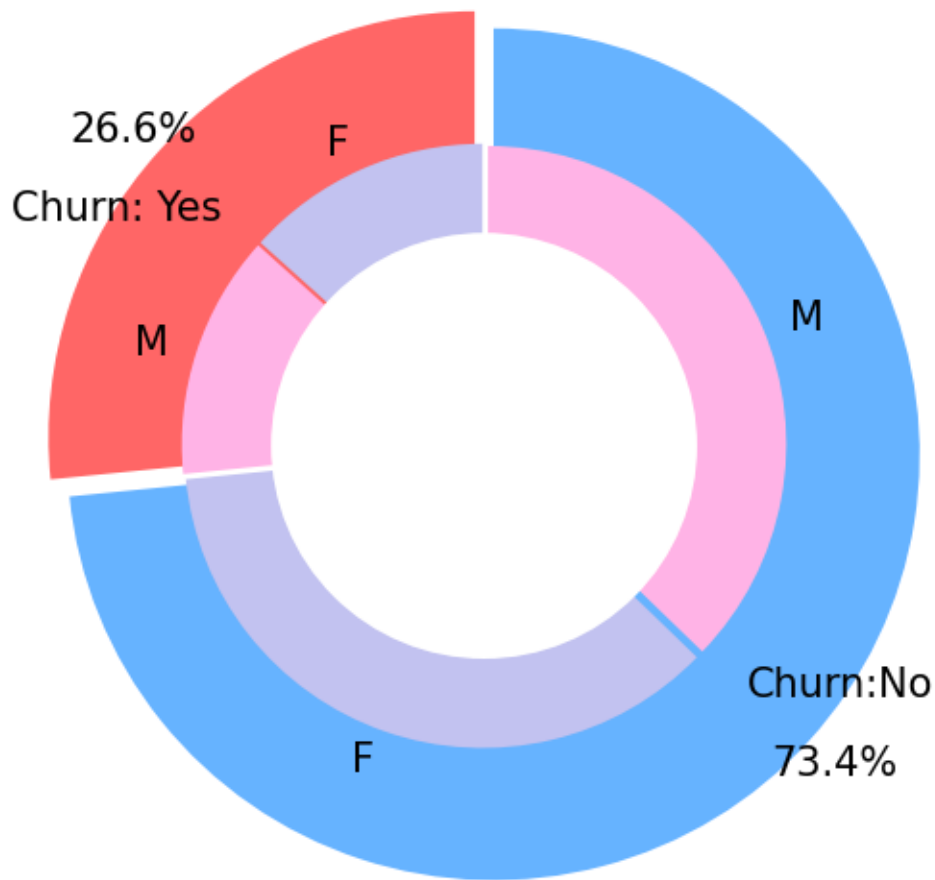
plt.title('Churn Distribution w.r.t Gender: Male(M), Female(F)', fontsize=15,
        ↪y=1.1)

# show plot

plt.axis('equal')
plt.tight_layout()
```

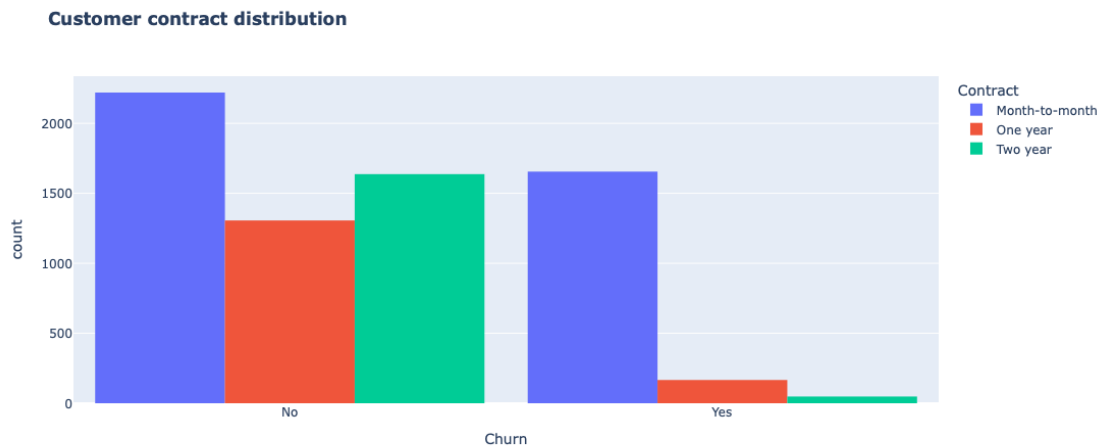
```
plt.show()
```

Churn Distribution w.r.t Gender: Male(M), Female(F)



- There is negligible difference in customer percentage/ count who changed the service provider. Both genders behaved in similar fashion when it comes to migrating to another service provider/firm.

```
[176]: fig = px.histogram(df, x="Churn", color="Contract", barmode="group",  
    title="Customer contract distribution")  
fig.update_layout(width=700, height=500, bargap=0.1)  
fig.show()
```



- About 75% of customer with Month-to-Month Contract opted to move out as compared to 13% of customers with One Year Contract and 3% with Two Year Contract

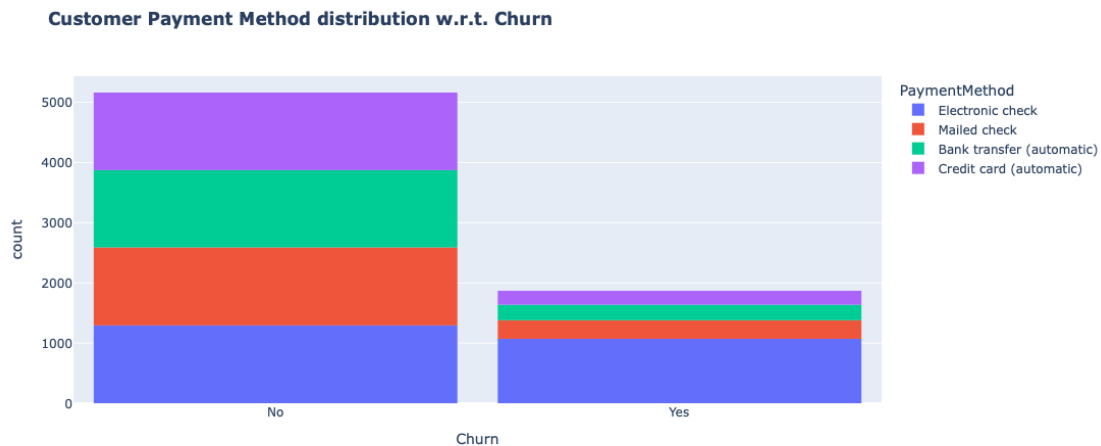
```
[179]: labels = df['PaymentMethod'].unique()
values = df['PaymentMethod'].value_counts()

fig = go.Figure(data=[go.Pie(labels=labels, values=values, hole=.3)])
fig.update_layout(title_text="<b>Payment Method Distribution</b>")
fig.show()
```

Payment Method Distribution



```
[181]: fig = px.histogram(df, x="Churn", color="PaymentMethod", title="<b>Customer_
Payment Method distribution w.r.t. Churn</b>")
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```

- Major customers who moved out were having Electronic Check as Payment Method.
- Customers who opted for Credit-Card automatic transfer or Bank Automatic Transfer and Mailed Check as Payment Method were less likely to move out.

```
[184]: df["InternetService"].unique()
```

```
[184]: array(['DSL', 'Fiber optic', 'No'], dtype=object)
```

```
[186]: df[df["gender"]=="Male"][["InternetService", "Churn"]].value_counts()
```

```
[186]: InternetService  Churn
DSL                No      992
Fiber optic        No      910
No                 No      717
Fiber optic        Yes      633
DSL                Yes      240
No                 Yes       57
Name: count, dtype: int64
```

```
[188]: df[df["gender"]=="Female"][["InternetService", "Churn"]].value_counts()
```

```
[188]: InternetService  Churn
DSL                No      965
Fiber optic        No      889
No                 No      690
Fiber optic        Yes      664
DSL                Yes      219
No                 Yes       56
Name: count, dtype: int64
```

```
[190]: fig = go.Figure()

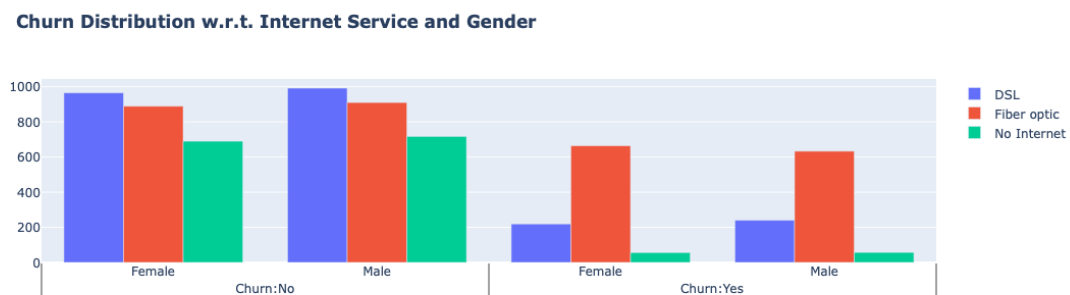
fig.add_trace(go.Bar(
    x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
         ["Female", "Male", "Female", "Male"]],
    y = [965, 992, 219, 240],
    name = 'DSL',
))

fig.add_trace(go.Bar(
    x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
         ["Female", "Male", "Female", "Male"]],
    y = [889, 910, 664, 633],
    name = 'Fiber optic',
))

fig.add_trace(go.Bar(
    x = [['Churn:No', 'Churn:No', 'Churn:Yes', 'Churn:Yes'],
         ["Female", "Male", "Female", "Male"]],
    y = [690, 717, 56, 57],
    name = 'No Internet',
))

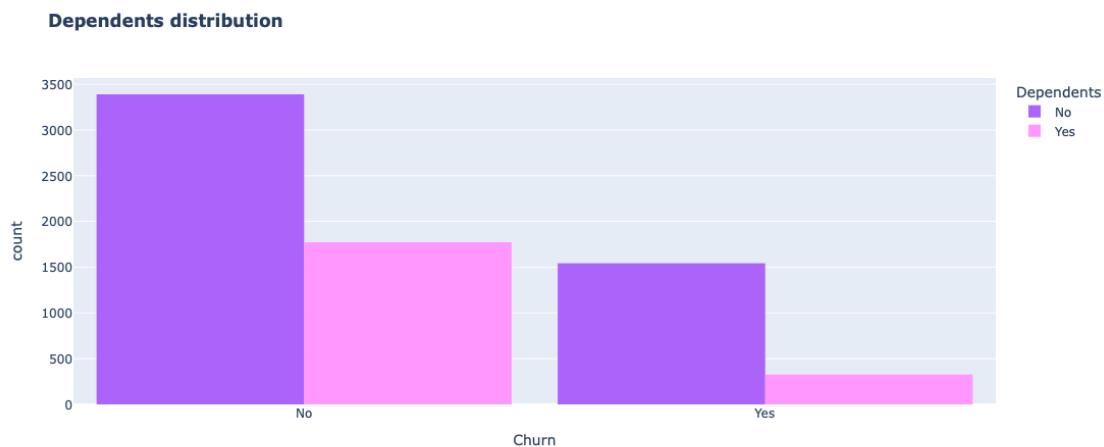
fig.update_layout(title_text="Churn Distribution w.r.t. Internet Service and Gender")

fig.show()
```



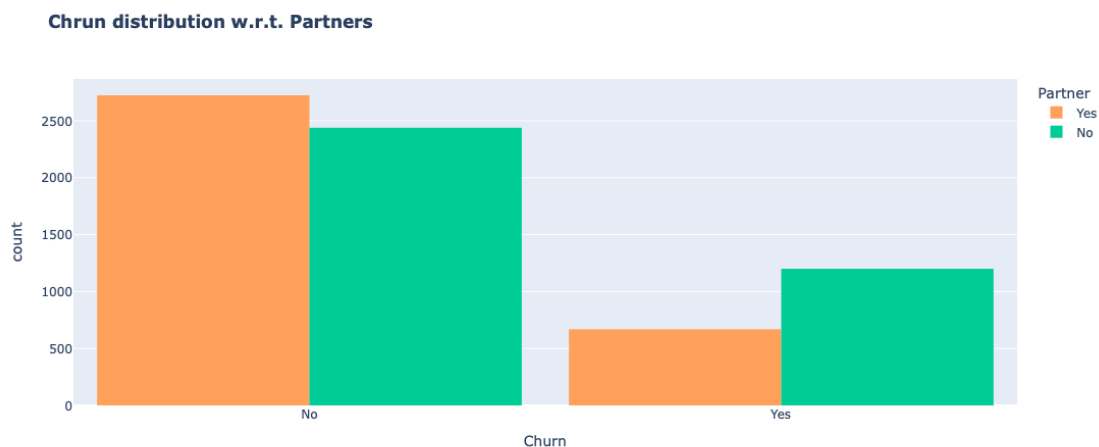
- A lot of customers choose the Fiber optic service and it's also evident that the customers who use Fiber optic have high churn rate, this might suggest a dissatisfaction with this type of internet service.
- Customers having DSL service are majority in number and have less churn rate compared to Fibre optic service.

```
[193]: color_map = {"Yes": "#FF97FF", "No": "#AB63FA"}
fig = px.histogram(df, x="Churn", color="Dependents", barmode="group",
    title="Dependents distribution", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



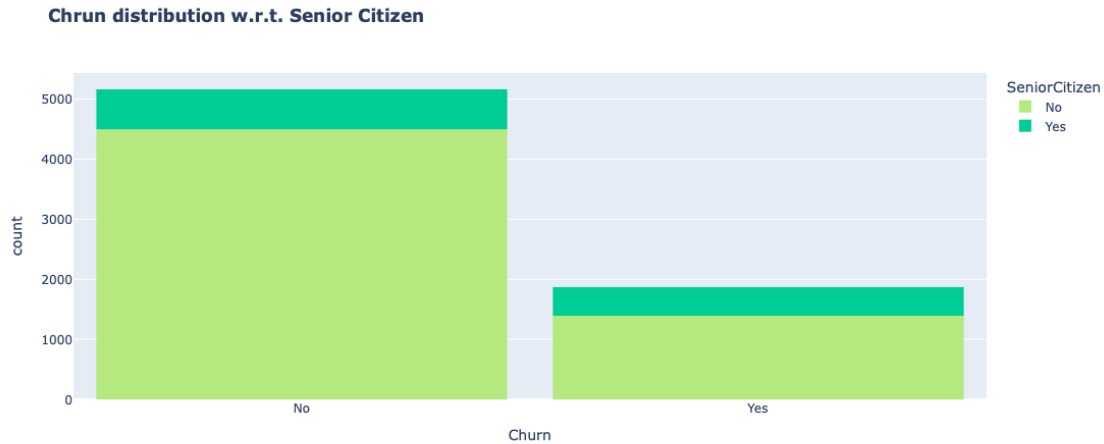
- Customers without dependents are more likely to churn

```
[196]: color_map = {"Yes": "#FFA15A", "No": "#00CC96"}
fig = px.histogram(df, x="Churn", color="Partner", barmode="group",
    title="Chrun distribution w.r.t. Partners",
    color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



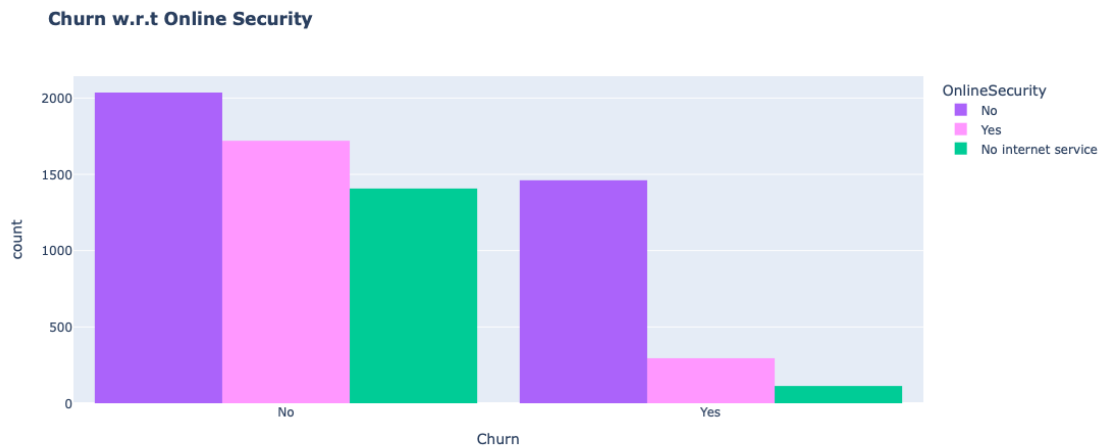
- Customers that doesn't have partners are more likely to churn

```
[199]: color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(df, x="Churn", color="SeniorCitizen", title="<b>Churn_
↳distribution w.r.t. Senior Citizen</b>", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



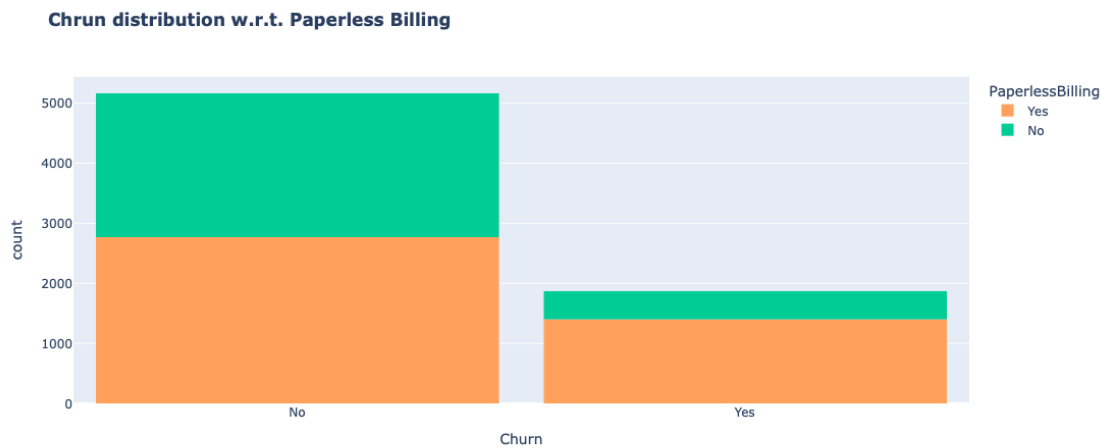
- It can be observed that the fraction of senior citizen is very less.
- Most of the senior citizens churn.

```
[202]: color_map = {"Yes": '#FF97FF', "No": '#AB63FA'}
fig = px.histogram(df, x="Churn", color="OnlineSecurity", barmode="group",
↳title="<b>Churn w.r.t Online Security</b>", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



- Most customers churn in the absence of online security,

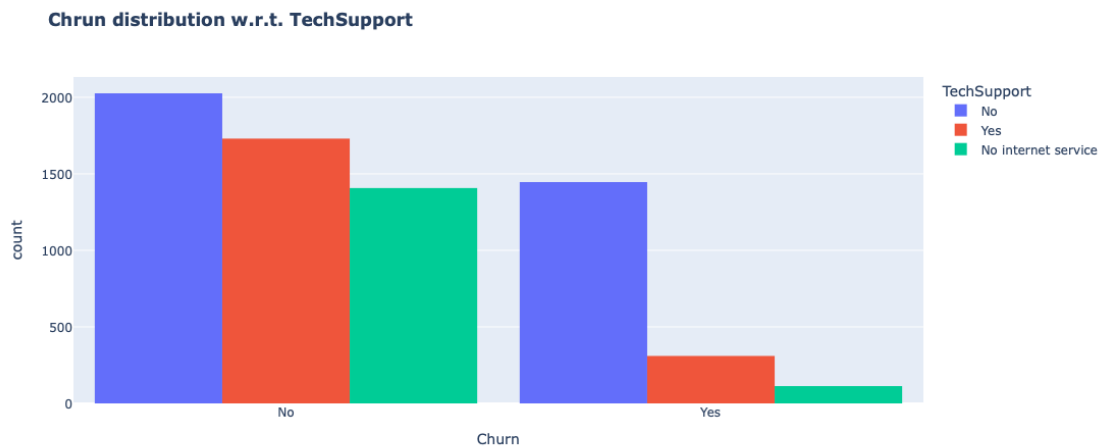
```
[205]: color_map = {"Yes": '#FFA15A', "No": '#00CC96'}
fig = px.histogram(df, x="Churn", color="PaperlessBilling", title="<b>Chrun_
distribution w.r.t. Paperless Billing</b>", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



- Customers with Paperless Billing are most likely to churn.

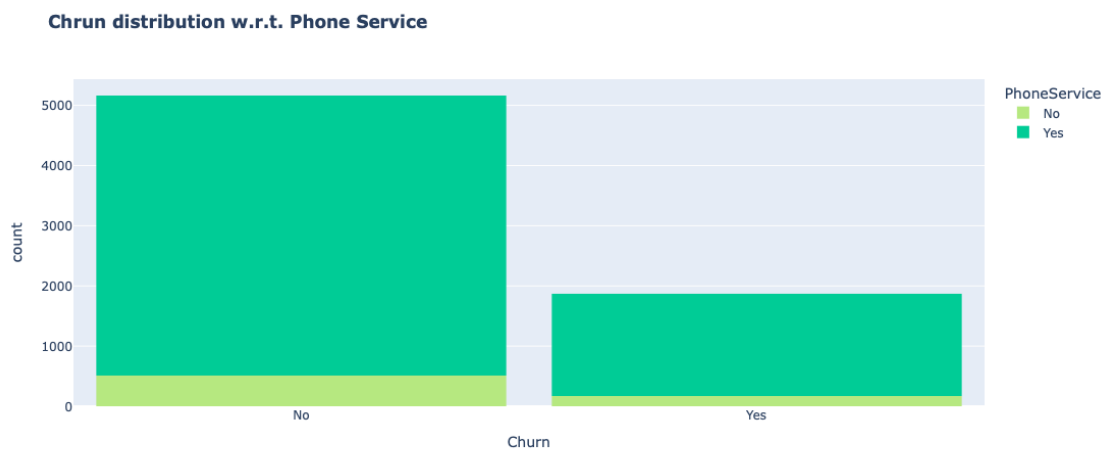
```
[208]: fig = px.histogram(df, x="Churn", color="TechSupport", barmode="group",
title="<b>Chrun distribution w.r.t. TechSupport</b>")
fig.update_layout(width=700, height=500, bargap=0.1)
```

```
fig.show()
```



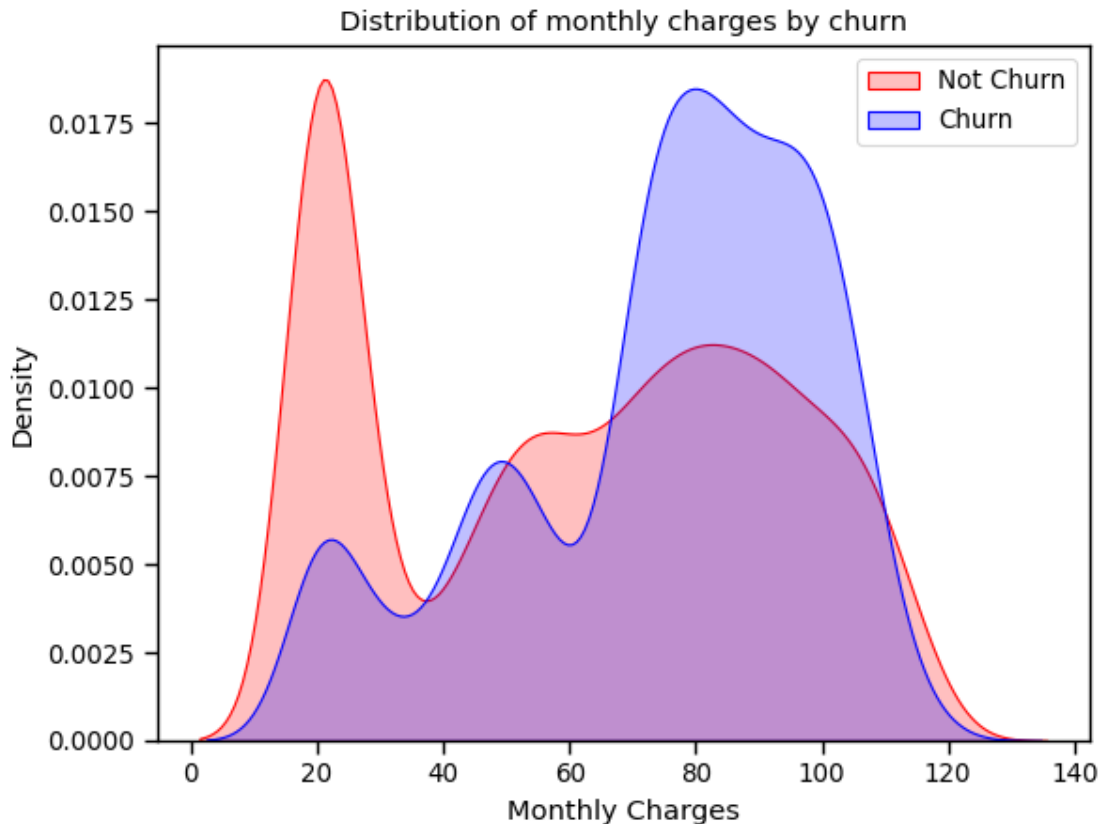
- Customers with no TechSupport are most likely to migrate to another service provider.

```
[211]: color_map = {"Yes": '#00CC96', "No": '#B6E880'}
fig = px.histogram(df, x="Churn", color="PhoneService", title="Chrun_
distribution w.r.t. Phone Service", color_discrete_map=color_map)
fig.update_layout(width=700, height=500, bargap=0.1)
fig.show()
```



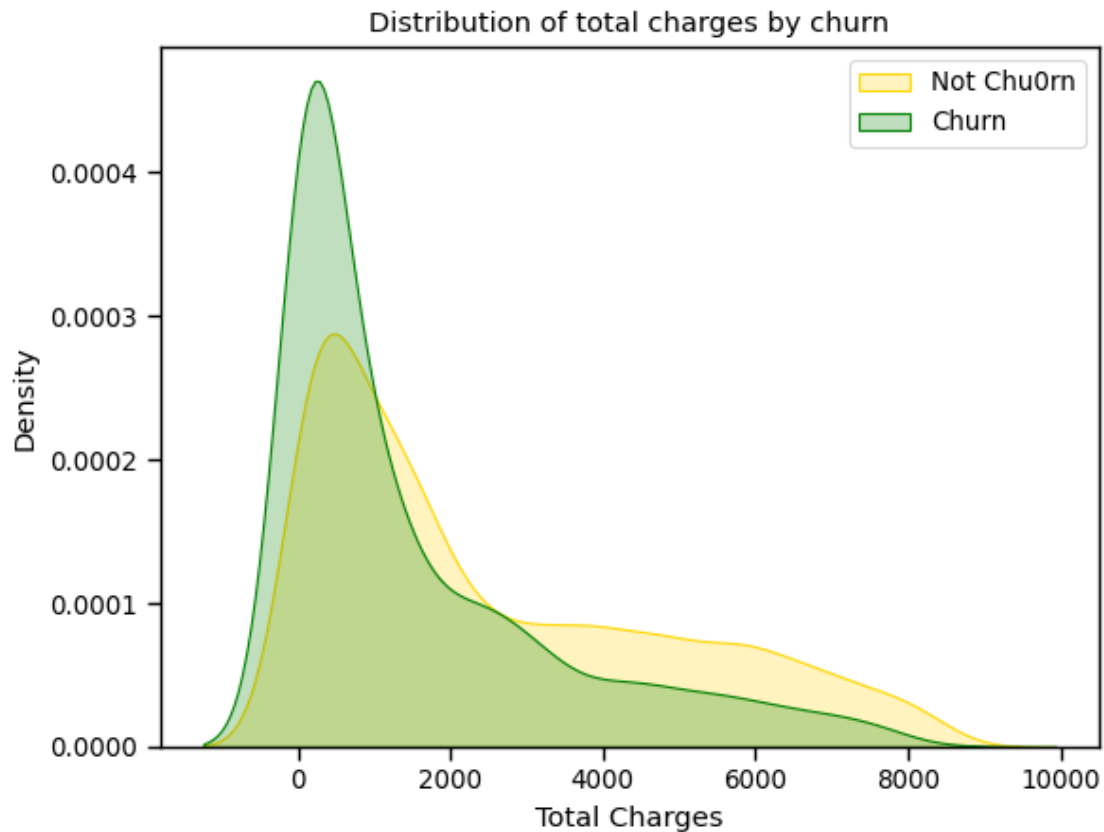
- Very small fraction of customers don't have a phone service and out of that, 1/3rd Customers are more likely to churn.

```
[214]: sns.set_context("paper",font_scale=1.1)
ax = sns.kdeplot(df.MonthlyCharges[(df["Churn"] == 'No') ],
                color="Red", shade = True);
ax = sns.kdeplot(df.MonthlyCharges[(df["Churn"] == 'Yes') ],
                ax=ax, color="Blue", shade= True);
ax.legend(["Not Churn","Churn"],loc='upper right');
ax.set_ylabel('Density');
ax.set_xlabel('Monthly Charges');
ax.set_title('Distribution of monthly charges by churn');
```



- Customers with higher Monthly Charges are also more likely to churn

```
[217]: ax = sns.kdeplot(df.TotalCharges[(df["Churn"] == 'No') ],
                    color="Gold", shade = True);
ax = sns.kdeplot(df.TotalCharges[(df["Churn"] == 'Yes') ],
                    ax=ax, color="Green", shade= True);
ax.legend(["Not Chu0rn","Churn"],loc='upper right');
ax.set_ylabel('Density');
ax.set_xlabel('Total Charges');
ax.set_title('Distribution of total charges by churn');
```

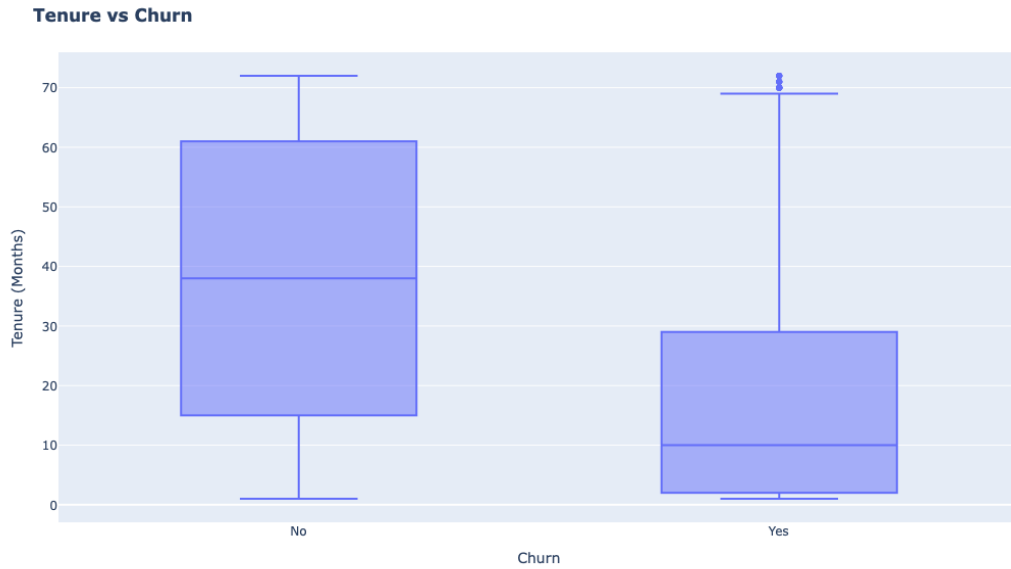


```
[219]: fig = px.box(df, x='Churn', y = 'tenure')

# Update yaxis properties
fig.update_yaxes(title_text='Tenure (Months)', row=1, col=1)
# Update xaxis properties
fig.update_xaxes(title_text='Churn', row=1, col=1)

# Update size and title
fig.update_layout(autosize=True, width=750, height=600,
                  title_font=dict(size=25, family='Courier'),
                  title='<b>Tenure vs Churn</b>',
)

fig.show()
```

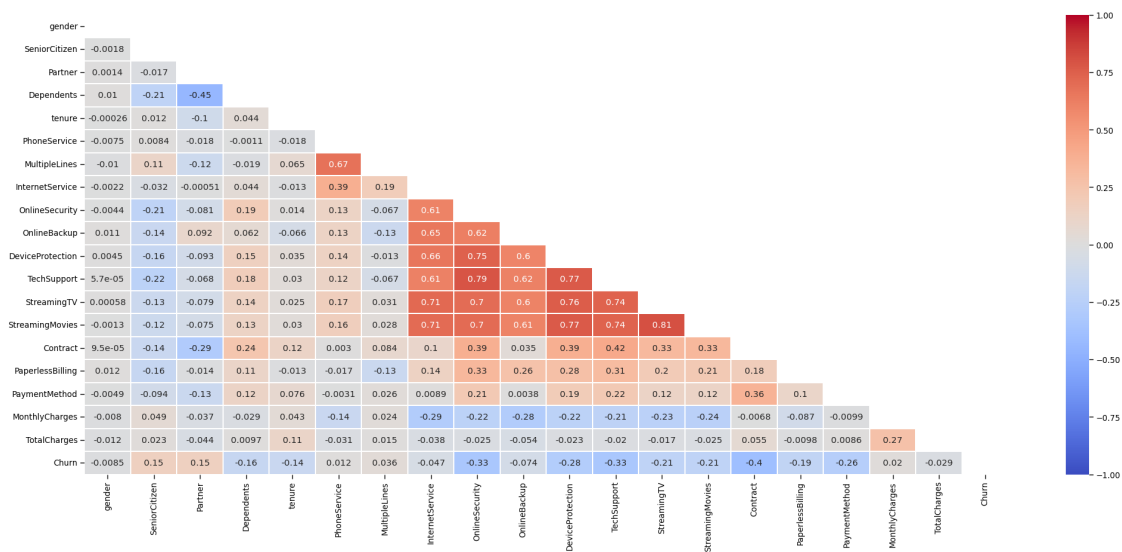
- New customers are more likely to churn

```
[222]: plt.figure(figsize=(25, 10))

corr = df.apply(lambda x: pd.factorize(x)[0]).corr()

mask = np.triu(np.ones_like(corr, dtype=bool))

ax = sns.heatmap(corr, mask=mask, xticklabels=corr.columns, yticklabels=corr.
    ↪columns, annot=True, linewidths=.2, cmap='coolwarm', vmin=-1, vmax=1)
```



7. Data Preprocessing

Splitting the data into train and test sets

```
[226]: def object_to_int(dataframe_series):  
        if dataframe_series.dtype=='object':  
            dataframe_series = LabelEncoder().fit_transform(dataframe_series)  
        return dataframe_series
```

```
[228]: df = df.apply(lambda x: object_to_int(x))  
df.head()
```

```
[228]: gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \  
0         0             0         1             0         1             0  \  
1         1             0         0             0        34             1  \  
2         1             0         0             0         2             1  \  
3         1             0         0             0        45             0  \  
4         0             0         0             0         2             1  \  
  
MultipleLines  InternetService  OnlineSecurity  OnlineBackup  \  
0              1                0                0              2  \  
1              0                0                2              0  \  
2              0                0                2              2  \  
3              1                0                2              0  \  
4              0                1                0              0  \  
  
DeviceProtection  TechSupport  StreamingTV  StreamingMovies  Contract  \  
0                 0            0            0              0          0  \  
1                 2            0            0              0          1  \  
2                 0            0            0              0          0  \  
3                 2            2            0              0          1  \  
4                 0            0            0              0          0  \  
  
PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges  Churn  \  
0                 1              2          29.85         29.85    0  \  
1                 0              3          56.95        1889.50    0  \  
2                 1              3          53.85         108.15    1  \  
3                 0              0          42.30        1840.75    0  \  
4                 1              2          70.70         151.65    1
```

```
[230]: plt.figure(figsize=(14,7))  
df.corr()['Churn'].sort_values(ascending = False)
```

```
[230]: Churn          1.000000  
MonthlyCharges    0.192858  
PaperlessBilling   0.191454  
SeniorCitizen      0.150541
```

```

PaymentMethod      0.107852
MultipleLines      0.038043
PhoneService       0.011691
gender             -0.008545
StreamingTV        -0.036303
StreamingMovies    -0.038802
InternetService    -0.047097
Partner            -0.149982
Dependents         -0.163128
DeviceProtection   -0.177883
OnlineBackup       -0.195290
TotalCharges       -0.199484
TechSupport        -0.282232
OnlineSecurity     -0.289050
tenure             -0.354049
Contract           -0.396150
Name: Churn, dtype: float64

```

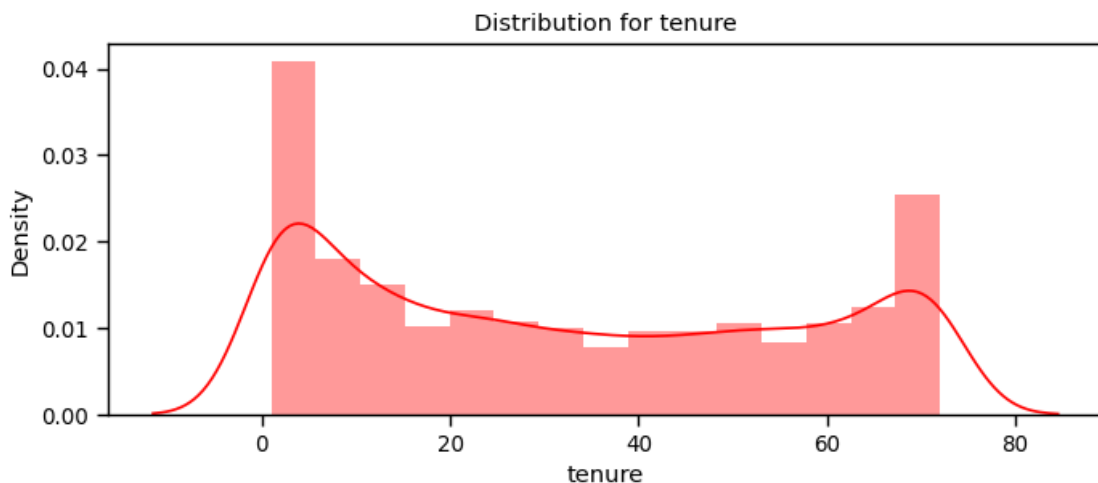
<Figure size 1400x700 with 0 Axes>

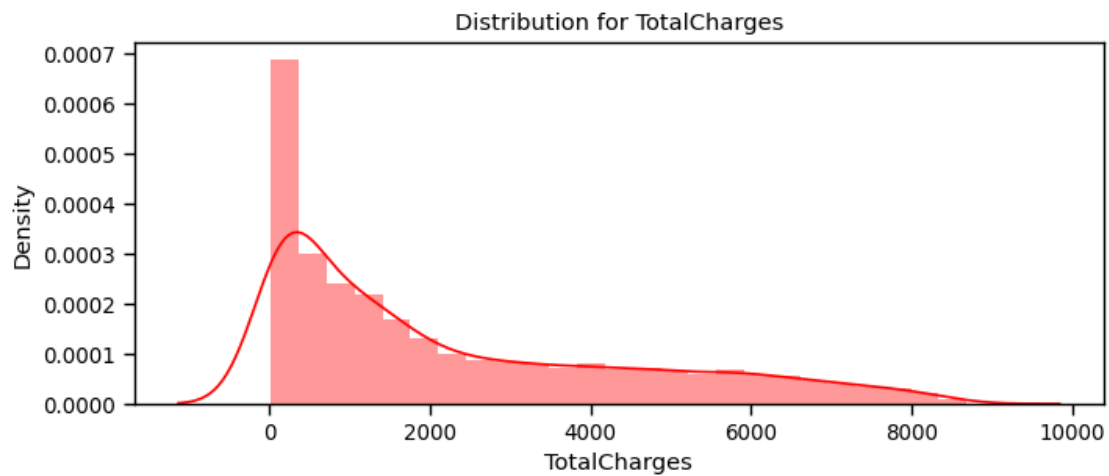
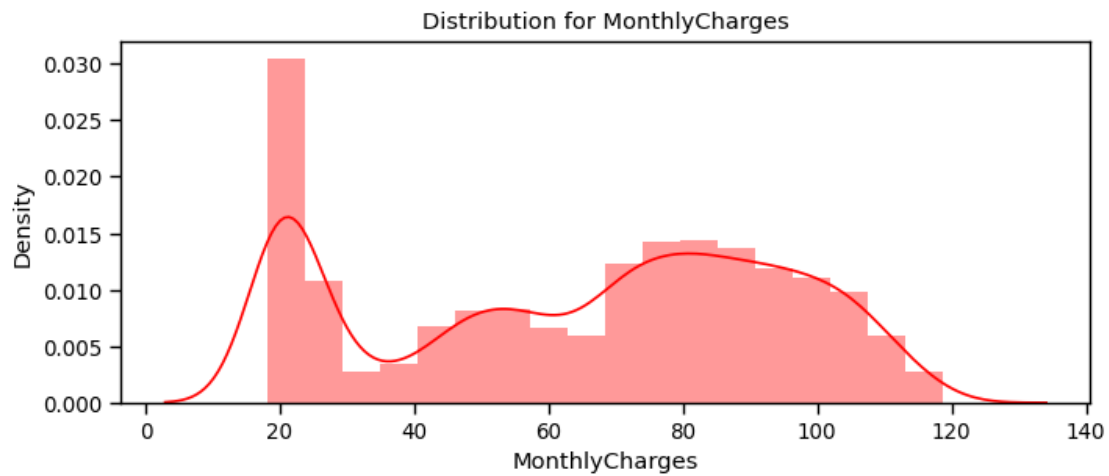
```
[232]: X = df.drop(columns = ['Churn'])
      y = df['Churn'].values
```

```
[234]: X_train, X_test, y_train, y_test = train_test_split(X,y,test_size = 0.30,
      ↪random_state = 40, stratify=y)
```

```
[236]: def distplot(feature, frame, color='r'):
      plt.figure(figsize=(8,3))
      plt.title("Distribution for {}".format(feature))
      ax = sns.distplot(frame[feature], color= color)
```

```
[238]: num_cols = ["tenure", 'MonthlyCharges', 'TotalCharges']
      for feat in num_cols: distplot(feat, df)
```

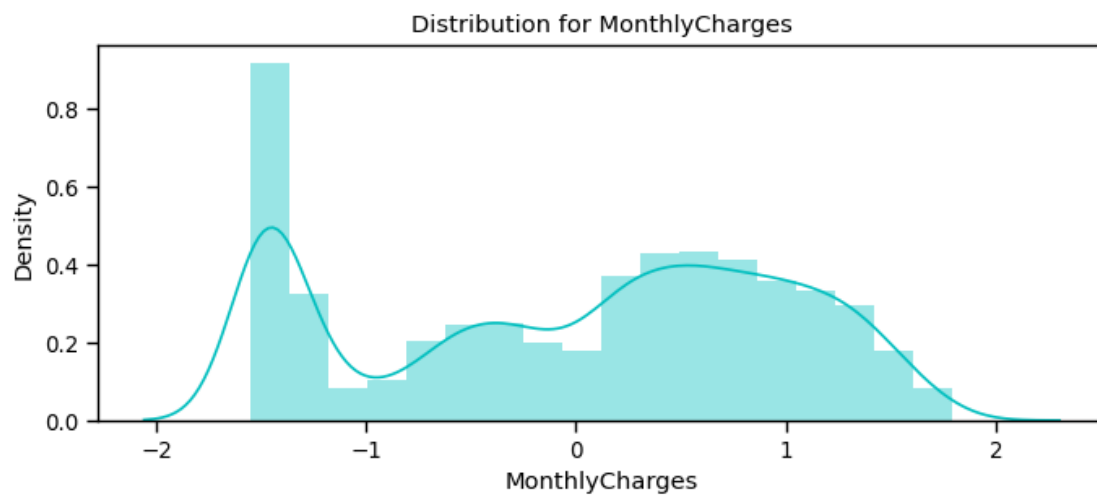
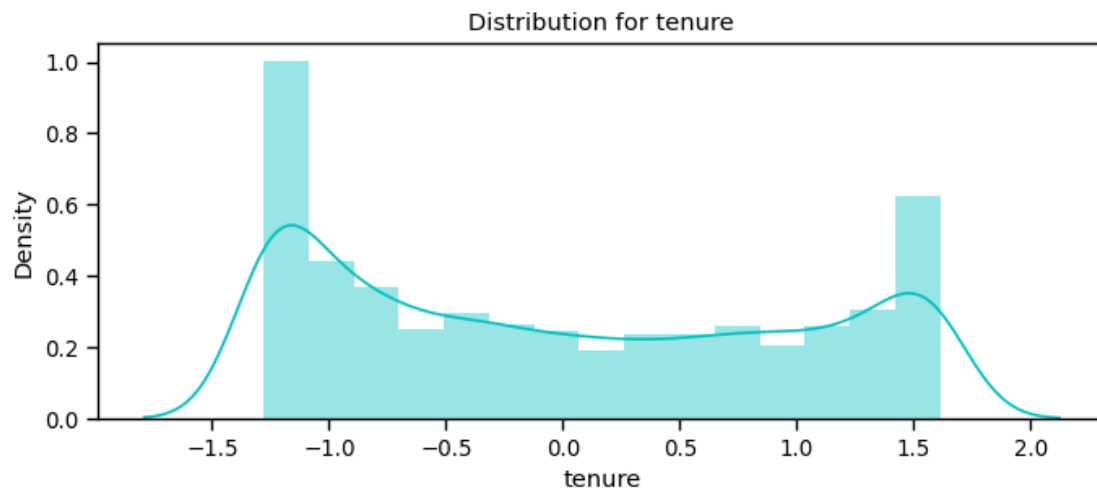


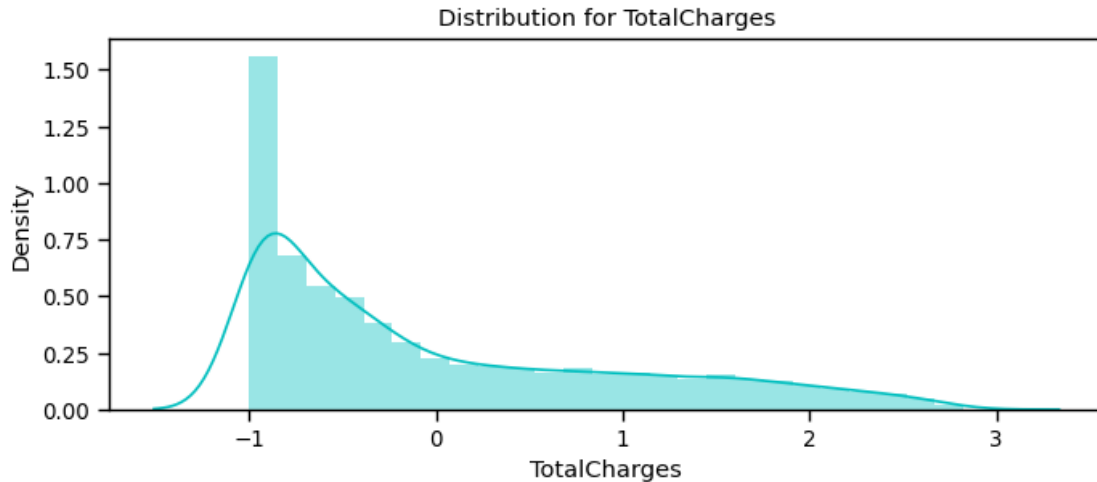


Since the numerical features are distributed over different value ranges, I will use standard scalar to scale them down to the same range.

Standardizing numeric attributes

```
[242]: df_std = pd.DataFrame(StandardScaler().fit_transform(df[num_cols]).
    ↳ astype('float64')),
    columns=num_cols)
for feat in numerical_cols: distplot(feat, df_std, color='c')
```





```
[243]: # Divide the columns into 3 categories, one for standardisation, one for label
        ↪ encoding and one for one hot encoding
```

```
cat_cols_ohe = ['PaymentMethod', 'Contract', 'InternetService'] # those that
        ↪ need one-hot encoding
cat_cols_le = list(set(X_train.columns) - set(num_cols) - set(cat_cols_ohe))
        ↪ #those that need label encoding
```

```
[246]: scaler= StandardScaler()

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

8. Machine Learning Model Evaluations and Predictions

KNN

```
[250]: knn_model = KNeighborsClassifier(n_neighbors = 11)
knn_model.fit(X_train,y_train)
predicted_y = knn_model.predict(X_test)
accuracy_knn = knn_model.score(X_test,y_test)
print("KNN accuracy:",accuracy_knn)
```

KNN accuracy: 0.776303317535545

```
[252]: print(classification_report(y_test, predicted_y))
```

	precision	recall	f1-score	support
0	0.83	0.87	0.85	1549
1	0.59	0.52	0.55	561

accuracy			0.78	2110
macro avg	0.71	0.69	0.70	2110
weighted avg	0.77	0.78	0.77	2110

SVC

```
[255]: svc_model = SVC(random_state = 1)
svc_model.fit(X_train,y_train)
predict_y = svc_model.predict(X_test)
accuracy_svc = svc_model.score(X_test,y_test)
print("SVM accuracy is :",accuracy_svc)
```

SVM accuracy is : 0.8075829383886256

```
[256]: print(classification_report(y_test, predict_y))
```

	precision	recall	f1-score	support
0	0.84	0.92	0.88	1549
1	0.69	0.50	0.58	561
accuracy			0.81	2110
macro avg	0.76	0.71	0.73	2110
weighted avg	0.80	0.81	0.80	2110

Logistic Regression

```
[258]: lr_model = LogisticRegression()
lr_model.fit(X_train,y_train)
accuracy_lr = lr_model.score(X_test,y_test)
print("Logistic Regression accuracy is :",accuracy_lr)
```

Logistic Regression accuracy is : 0.8090047393364929

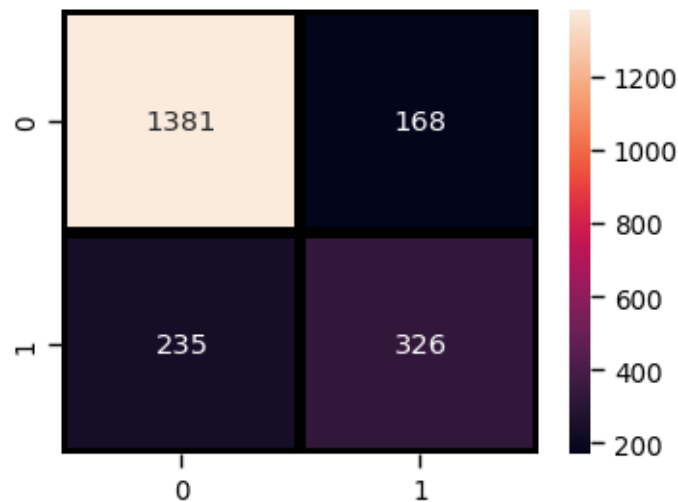
```
[259]: lr_pred= lr_model.predict(X_test)
report = classification_report(y_test,lr_pred)
print(report)
```

	precision	recall	f1-score	support
0	0.85	0.89	0.87	1549
1	0.66	0.58	0.62	561
accuracy			0.81	2110
macro avg	0.76	0.74	0.75	2110
weighted avg	0.80	0.81	0.80	2110

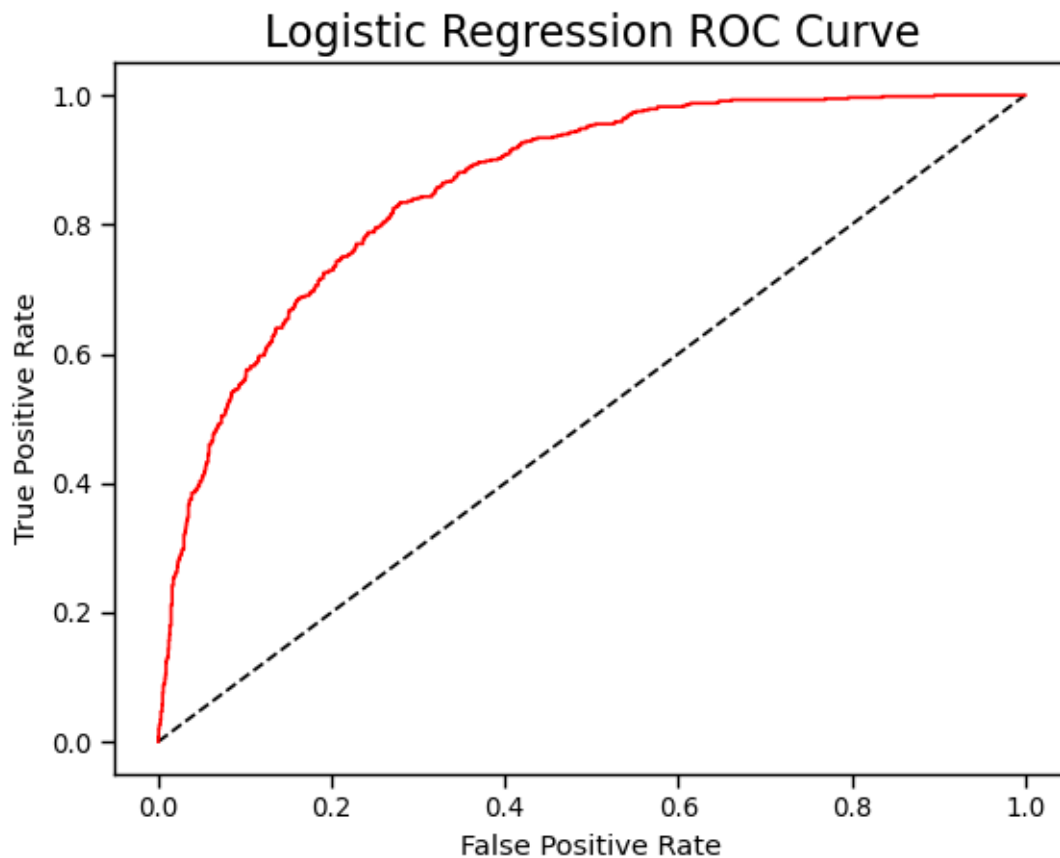
```
[264]: plt.figure(figsize=(4,3))
sns.heatmap(confusion_matrix(y_test, lr_pred),
            annot=True,fmt = "d",linecolor="k",linewidths=3)

plt.title("LOGISTIC REGRESSION CONFUSION MATRIX",fontsize=14)
plt.show()
```

LOGISTIC REGRESSION CONFUSION MATRIX



```
[266]: y_pred_prob = lr_model.predict_proba(X_test)[:,-1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
plt.plot([0, 1], [0, 1], 'k--' )
plt.plot(fpr, tpr, label='Logistic Regression',color = "r")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Logistic Regression ROC Curve',fontsize=16)
plt.show();
```

Decision Tree Classifier

```
[269]: dt_model = DecisionTreeClassifier()
dt_model.fit(X_train,y_train)
predictdt_y = dt_model.predict(X_test)
accuracy_dt = dt_model.score(X_test,y_test)
print("Decision Tree accuracy is :",accuracy_dt)
```

Decision Tree accuracy is : 0.723696682464455

Decision tree gives very low score.

```
[272]: print(classification_report(y_test, predictdt_y))
```

	precision	recall	f1-score	support
0	0.82	0.80	0.81	1549
1	0.48	0.52	0.50	561
accuracy			0.72	2110
macro avg	0.65	0.66	0.65	2110

weighted avg 0.73 0.72 0.73 2110

Gradient Boosting Classifier

```
[275]: gb = GradientBoostingClassifier()
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)
print("Gradient Boosting Classifier", accuracy_score(y_test, gb_pred))
```

Gradient Boosting Classifier 0.8075829383886256

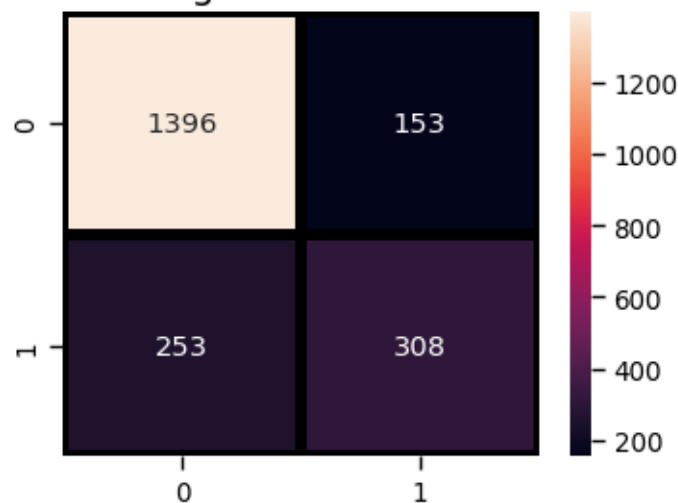
```
[276]: print(classification_report(y_test, gb_pred))
```

	precision	recall	f1-score	support
0	0.85	0.90	0.87	1549
1	0.67	0.55	0.60	561
accuracy			0.81	2110
macro avg	0.76	0.73	0.74	2110
weighted avg	0.80	0.81	0.80	2110

```
[277]: plt.figure(figsize=(4,3))
sns.heatmap(confusion_matrix(y_test, gb_pred),
            annot=True,fmt = "d",linecolor="k",linewidths=3)

plt.title("Gradient Boosting Classifier Confusion Matrix",fontsize=14)
plt.show()
```

Gradient Boosting Classifier Confusion Matrix



Customer churn is definitely bad to a firm 's profitability. Various strategies can be implemented to eliminate customer churn.

The best way to avoid customer churn is for a company to truly know its customers.

This includes identifying customers who are at risk of churning and working to improve their satisfaction. Improving customer service is, of course, at the top of the priority for tackling this issue. Building customer loyalty through relevant experiences and specialized service is another strategy to reduce customer churn.

Some firms survey customers who have already churned to understand their reasons for leaving in order to adopt a proactive approach to avoiding future customer churn.

[]: